TraceBack — Concurrency Test Report

6 Nov 2026 · companion to the TraceBack Test Reports

Result: all 14 concurrency checks passed. Each race was repeated 40 times (a run of 10 and a run of 30) with no failure, no deadlock and no server error (500).

When two people act on the same item or claim at the same moment, they go one after the other: the first action is recorded, and the second is refused with a message instead of being recorded twice. All 459 unit and integration tests and all 15 browser system tests also pass.

TraceBack is the IIT Kanpur Lost & Found portal. Concurrency testing checks what happens when requests overlap: two guards pressing the same button, a student double-clicking, or several background workers running at once.

1. Approach

- **Where:** the full app (API, worker, website, PostgreSQL 16) on a fresh test database, on a 2-CPU machine like the t3.small server. The live site was not touched.
- **How:** each race fires its requests at the same instant, using a thread barrier and a separate connection per request, the way two people in different places would. A race only shows when requests really overlap, so each one was repeated on 40 fresh items or claims: a run of 10, then a run of 30.
- **What was checked:** the database afterwards, meaning claim and custody history, codes stored, photos and AI jobs, as well as the HTTP responses, the API log and the PostgreSQL log (for deadlocks).
- **Script:** `testing/concurrency_tests.py`; raw results in `testing/results/` (see section 6).

2. Results

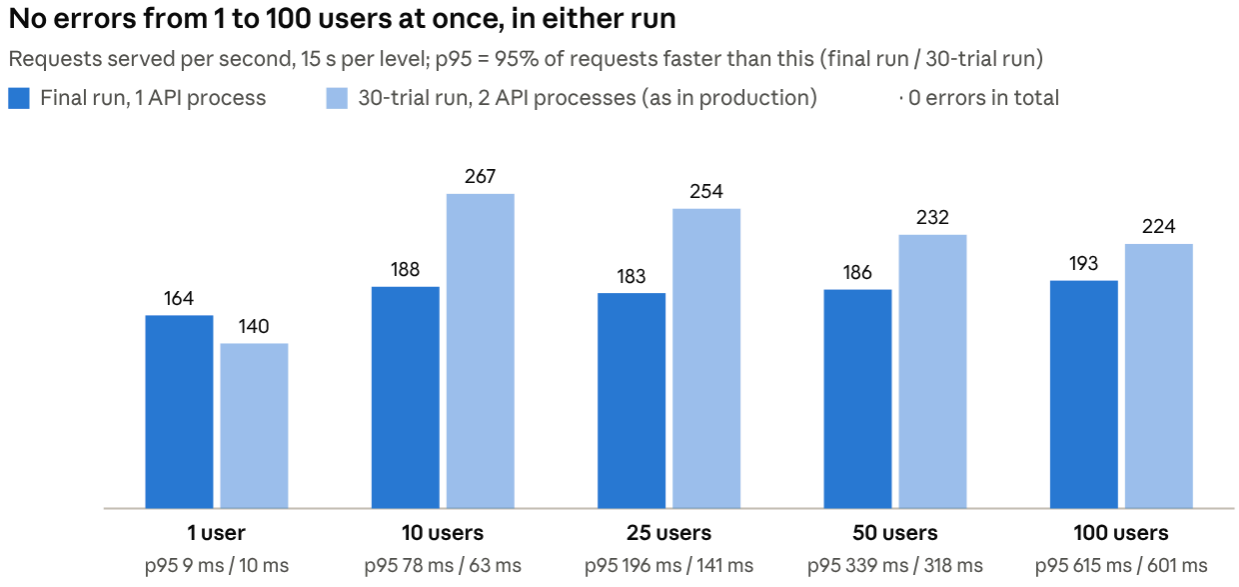
ID	Scenario	Expected	Observed (40 trials unless noted)	Result
CON-01	10 simultaneous sign-in code requests for one address	1 code sent, 9 refused	1 code sent every time; the other 9 were told to wait (429)	Pass

ID	Scenario	Expected	Observed (40 trials unless noted)	Result
CON-02	20 simultaneous wrong guesses on one code	At most 5 counted; code locked	Exactly 5 counted every time; the right code was refused afterwards	Pass
CON-03	The correct code sent 5 times at once	1 sign-in	Exactly 1 sign-in every time	Pass
CON-04	Two guards receive the same item at once, at two offices	1 succeeds; 1 receipt in the history	1 receipt every time; the second guard was refused	Pass
CON-05	A student sends the same claim 5 times at once	1 claim; duplicates refused with 409	1 claim every time; all 160 duplicates were refused with 409	Pass
CON-06	Two guards approve two different claims on one item	Only 1 approved	Only 1 approved every time; the second guard got 409	Pass
CON-07	One guard approves while another rejects the same claim	1 decision	1 decision every time; the second guard was refused	Pass
CON-08	Two guards hand over the same item at once	1 hand-over	1 “collected” and 1 “returned” entry every time; the second guard was refused	Pass
CON-09	Claimant withdraws as a guard hands the item over	Either returned or still held, with a matching history	23 times withdrawn and still held, 17 times collected and returned; the history always matched	Pass
CON-10	6 photos uploaded at once to an item limited to 3	At most 3 stored	3 stored every time	Pass
CON-11	Two guards move the same item to different offices	Office matches the newest history entry	Matched every time	Pass

ID	Scenario	Expected	Observed (40 trials unless noted)	Result
CON-12	5 copies of the worker send reminders at once	1 reminder per item	1 reminder per item every time	Pass
CON-13	30 reports at once, AI queue served by 3 runners (1 trial per run)	Each job done once, no duplicate matches	30 of 30 jobs done once in both runs; no duplicate suggestions	Pass
CON-14	1 to 100 students browsing at once, 15 s per level (5 levels per run)	No errors; 95% under 1 s at 50 users	No errors at any level; at 50 users 95% under 0.33 s, at 100 users under 0.6 s	Pass

In both runs, the API log showed no server errors and the PostgreSQL log no deadlocks. The final run (10 trials) was made on the finished code, after the security and recovery fixes; the 30-trial run repeated every race on the same locking code.

3. Load test



testing/concurrency_tests.py (CON-14), 15 s per level, local copy on a 2-CPU machine like the t3.small · testing/results/concurrency_results.json and concurrency_results_30_trials.json

From 1 to 100 simultaneous users, every request was answered in both runs. With two API processes, as on the server, throughput reached about 270 requests a second at 10 users and stayed between 224 and 254 from 25 to 100 users, as requests queued for the 2 CPUs. The final run used a single API process and served 164–193 requests a second. At 100

users, 95% of requests were answered within 601–615 ms. Each API process works on at most 30 requests at once and has a database connection for each, so no request waits for a connection; extra requests queue without holding anything.

4. What protects the app

- **One action at a time per item:** every claim action (approve, reject, start or resume review, office visit, withdraw, hand-over, expiry) locks the item's row and then the claim's row, and reads both again before checking whether the action is allowed (`claims._lock`). A second action at the same moment waits, sees the new status and is refused. Custody actions (receive, move, dispose) lock the item the same way (`custody._lock`).
- **Always the same lock order:** item first, then claim, in every action, so two requests can never each hold a lock the other is waiting for, and no deadlock can occur.
- **Sign-in codes:** a request for a code first takes a lock on that email address (`pg_advisory_xact_lock`), so parallel requests see each other's code and respect the 60-second wait. The code row is locked while it is checked (`FOR UPDATE`), so parallel guessing can't skip the 5-try limit.
- **Claims and photos:** a new claim locks its item, and a duplicate that still reaches the database's "one open claim per person" rule is answered with 409. A photo upload locks the item while it counts the photos, so the limit of 3 holds.
- **Database rules as a second guard:** unique indexes allow only one approved claim per item and one open claim per person per item, and a check constraint stops an item marked "held" with no office.
- **Background work:** reminders and AI jobs are taken with `FOR UPDATE SKIP LOCKED`, so several workers never do the same job twice.
- **Load:** at most 30 requests at once per API process, with a database connection for each, and two API processes.

5. Observations

The request that loses a race gets the standard refusal message, for example "A claim that is collected can't be moved to collected by you" (403), or 409 for a second approval. A clearer "another guard has just done this" message would be a small improvement.

6. How to re-run

Start a fresh copy with `bash testing/start_stack.sh`, then run `python testing/concurrency_tests.py` (about 5 minutes). Set `PSQL` if your PostgreSQL login differs from `psql -h localhost -U postgres`, and `TRIALS` to repeat each race more or fewer times (default 10; the second run used `TRIALS=30`, about 12 minutes). The script needs Linux, macOS or WSL.

Raw results in `testing/results/`: `concurrency_results.json` (run of 10) and `concurrency_results_30_trials.json` (run of 30), with the run logs `concurrency_run.log` and `concurrency_run_30_trials.log`.